Red Hat OpenShift AI Self-Managed 2.25

Working with connected applications

Connect to applications from Red Hat OpenShift AI Self-Managed

Last Updated: 2025-10-28

Red Hat OpenShift AI Self-Managed 2.25 Working with connected applications

Connect to applications from Red Hat OpenShift AI Self-Managed

Legal Notice

Abstract

Learn how to enable access to connected applications, remove unused applications from your dashboard, and access and use the Jupyter application that is enabled by default.

Table of Contents

PREFACE

You can extend Red Hat OpenShift AI capabilities by connecting to a wide range of open source and third-party applications, such as Starburst and IBM watsonx.ai.

You can also remove unused applications from your OpenShift AI dashboard so that you can focus on the applications that you are most likely to use.

CHAPTER 1. VIEWING APPLICATIONS THAT ARE CONNECTED TO OPENSHIFT AI

You can view the available open source and third-party connected applications from the OpenShift AI dashboard.

Prerequisites

- You have logged in to Red Hat OpenShift AI.

Procedure

1. From the OpenShift AI dashboard, select **Applications → Explore**.
   The **Explore** page displays applications that are available for use with OpenShift AI.
2. Click a tile for more information about the application or to access the **Enable** button.
   Note: The **Enable** button is visible only if an application does not require an OpenShift Operator installation.

Verification

You can access the **Explore** page and click on tiles.

CHAPTER 2. ENABLING APPLICATIONS THAT ARE CONNECTED TO OPENSHIFT AI

You must enable SaaS-based applications before using them with Red Hat OpenShift AI. On-cluster applications are enabled automatically.

Typically, you can install, or enable applications connected to OpenShift AI using one of the following methods:

- Enabling the application from the **Explore** page on the OpenShift AI dashboard, as documented in the following procedure.
- Installing the Operator for the application from OperatorHub. OperatorHub is a web console for cluster administrators to discover and select Operators to install on their cluster. It is deployed by default in OpenShift. For more information, see Installing from OperatorHub using the web console.

NOTE

Deployments containing Operators installed from OperatorHub may not be fully supported by Red Hat.

- Installing the application as an Operator to your cluster. For more information, see Adding Operators to a cluster.

For some applications (such as Jupyter), the API endpoint is available on the tile for the application on the **Enabled** page of OpenShift AI. Certain applications cannot be accessed directly from their tiles, for example, OpenVINO provides notebook images for use in Jupyter and does not provide an endpoint link from its tile. Additionally, it might be useful to store these endpoint URLs as environment variables for easy reference in a notebook environment.

Some independent software vendor (ISV) applications must be installed in specific namespaces. In these cases, the tile for the application in the OpenShift AI dashboard specifies the required namespace.

To help you get started quickly, you can access the application's learning resources and documentation on the **Resources** page, or on the **Enabled** page by clicking the relevant link on the tile for the application.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- Your administrator has installed or configured the application on your OpenShift cluster.

Procedure

1. On the OpenShift AI home page, click **Explore**.
2. On the **Explore** page, find the tile for the application that you want to enable.
3. Click **Enable** on the application tile.
4. If prompted, enter the application's service key and then click **Connect**.
5. Click **Enable** to confirm that you want to enable the application.

Verification

- The application that you enabled is displayed on the **Enabled** page.
- The API endpoint is displayed on the tile for the application on the **Enabled** page.

CHAPTER 3. REMOVING DISABLED APPLICATIONS FROM THE DASHBOARD

After your administrator has disabled your unused applications, you can manually remove them from the Red Hat OpenShift AI dashboard. Disabling and removing unused applications allows you to focus on the applications that you are most likely to use.

Prerequisites

- You are logged in to Red Hat OpenShift AI.
- Your administrator has disabled the application that you want to remove, as described in Disabling applications connected to OpenShift AI .

Procedure

1. In the OpenShift AI interface, click **Applications → Enabled**.
   On the **Enabled** page, tiles for disabled applications are denoted with a **Disabled** label.
2. Click **Disabled** on the tile for the application that you want to remove.
3. Click the link to remove the application tile.

Verification

- The tile for the disabled application is no longer displayed on the **Enabled** page.

CHAPTER 4. USING BASIC WORKBENCHES

Red Hat OpenShift AI provides access to **Start basic workbench** as an enabled application for situations where, for example, you do not want users to have their own data science projects or you want to open a Jupyter notebook that was developed outside of OpenShift AI and has no dependencies on other environments.

Note that the preferred way to access workbenches on OpenShift AI is through a data science project, as described in Creating a workbench and selecting an IDE . The advantages to using an OpenShift AI data science project and creating a workbench that includes Jupyter, is that your project organizes your data science work in one place and adds functionality such as connections so that you can access data and save your models and pipelines for automating your ML workflow.

4.1. STARTING A BASIC WORKBENCH

Using the basic workbench is based on a server-client architecture. The basic workbench runs in a container on the Red Hat OpenShift cluster. The client is the IDE interface that opens in your web browser on your local computer. However, all of the commands that you enter in the IDE are executed by the workbench. This architecture allows you to interact through your local computer in a browser environment, while all processing occurs on the cluster. The cluster provides the benefits of larger available resources and security because the data being processed never leaves the cluster.

From the **Start basic workbench** tile, you can start a basic workbench. If you require extra power for use with large datasets, you can assign accelerators to your workbench to optimize performance.

Prerequisites

- You are logged in to Red Hat OpenShift AI.
- You are starting the workbench for the first time or you stopped your workbench.
- You know the names and values you want to use for any environment variables in your workbench environment, for example, **AWS_SECRET_ACCESS_KEY**.
- If you want to work with a large data set, work with your administrator to proactively increase the storage capacity of your workbench. If applicable, also consider assigning accelerators to your workbench.

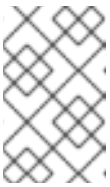
Procedure

1. In the left navigation pane, click **Applications → Enabled**.
2. On the **Enabled** page, locate the **Start basic workbench** tile.
3. Click **Open application**.
   If you see an **Access permission needed** message, you are not in the default user group or the default administrator group for OpenShift AI. Ask your administrator to add you to the correct group by using Adding users to OpenShift AI user groups .

   If your credentials are accepted, the **Workbench control panel** opens displaying the **Start a basic workbench** page.
4. Start a basic workbench.
   a. In the **Workbench image** section, select the workbench image to use for your workbench.

Different workbench images have different packages installed by default. Click the help icon (?) next to a workbench image name to view a list of its included packages.

b. If the workbench image contains multiple versions, select the version of the workbench image from the **Versions** section.



NOTE

When a new version of a workbench image is released, the previous version remains available and supported on the cluster. This gives you time to migrate your work to the latest version of the workbench image.

c. From the **Container size** list, select a suitable container size for your workbench.

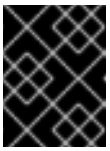
d. Optional: From the **Accelerator** list, select an accelerator.

e. If you selected an accelerator in the preceding step, specify the number of accelerators to use.

IMPORTANT

Using accelerators is only supported with specific workbench images. For GPUs, only the AMD ROCm, PyTorch, TensorFlow, and CUDA workbench images are supported. In addition, you can only specify the number of accelerators required for your workbench if accelerators are enabled on your cluster. To learn how to enable accelerator support, see Working with accelerators.

f. Optional: Select and specify values for any new **Environment variables**.
The interface stores these variables so that you only need to enter them once. Example variable names for common environment variables are automatically provided for frequently integrated environments and frameworks, such as Amazon Web Services (AWS).



IMPORTANT

Select the **Secret** checkbox for variables with sensitive values that must remain private, such as passwords.

g. Optional: Check **Start workbench in current tab**

h. Click **Start workbench**.
The **Workbench status** progress indicator is displayed. Click the **Events log** tab to view additional information about the workbench creation process. Depending on the deployment size and resources you requested, starting the workbench can take up to several minutes. Only click **Cancel** if you want to cancel the workbench creation.

After the server starts, you see one of the following behaviors:

- If you selected **Start workbench in current tab** in the preceding step, the IDE interface opens in the current tab of your web browser.
- If you did not select **Start workbench in current tab** in the preceding step, the **Workbench status** dialog box prompts you to open the server in a new browser tab or in the current browser tab.

Verification

- The IDE interface opens.

Troubleshooting

- If you see the "Unable to load workbench configuration options" error message, contact your administrator so that they can review the logs associated with your workbench pod and determine further details about the problem.

4.2. CREATING AND IMPORTING JUPYTER NOTEBOOKS

You can create a blank Jupyter notebook or import a Jupyter notebook in JupyterLab from several different sources.

4.2.1. Creating a Jupyter notebook

You can create a Jupyter notebook from an existing notebook container image to access its resources and properties. The **Workbench control panel** contains a list of available container images that you can run as a single-user workbench.

Prerequisites

- Ensure that you have logged in to Red Hat OpenShift AI.
- Ensure that you have launched your workbench and logged in to JupyterLab.
- The workbench image exists in a registry, image stream, and is accessible.

Procedure

1. Click **File → New → Notebook**.
2. If prompted, select a kernel for your Jupyter notebook from the list.
   If you want to use a kernel, click **Select**. If you do not want to use a kernel, click **No Kernel**.

Verification

- Check that the notebook file is visible in the JupyterLab interface.

4.2.2. Uploading an existing notebook file to JupyterLab from local storage

You can load an existing notebook file from local storage into JupyterLab to continue work, or adapt a project for a new use case.

Prerequisites

- Credentials for logging in to JupyterLab.
- You have a launched and running workbench based on a JupyterLab image.
- A notebook file exists in your local storage.

Procedure

1. In the **File Browser** in the left sidebar of the JupyterLab interface, click **Upload Files** ().
2. Locate and select the notebook file and then click **Open**.
   The file is displayed in the **File Browser**.

Verification

- The notebook file is displayed in the **File Browser** in the left sidebar of the JupyterLab interface.
- You can open the notebook file in JupyterLab.

4.2.3. Additional resources

- Collaborating on Jupyter notebooks by using Git

4.3. COLLABORATING ON JUPYTER NOTEBOOKS BY USING GIT

If your files are stored in Git version control, you can clone a Git repository to work with them in JupyterLab. When you are ready, you can push your changes back to the Git repository so that others can review or use your models.

4.3.1. Uploading an existing notebook file from a Git repository by using JupyterLab

You can use the JupyterLab user interface to clone a Git repository into your workspace to continue your work or integrate files from an external project.

Prerequisites

- You have a launched and running workbench based on a JupyterLab image.
- Read access for the Git repository you want to clone.

Procedure

1. Copy the HTTPS URL for the Git repository.
   - In GitHub, click **Code → HTTPS** and then click the **Copy URL to clipboard** icon.
   - In GitLab, click **Code** and then click the **Copy URL** icon under **Clone with HTTPS**.
2. In the JupyterLab interface, click the **Git Clone** button ().

   You can also click **Git → Clone a repository** in the menu, or click the Git icon () and click the **Clone a repository** button.

   The **Clone a repo** dialog opens.
3. Enter the HTTPS URL of the repository that contains your notebook file.
4. Click **CLONE**.
5. If prompted, enter your username and password for the Git repository.

Verification

- Check that the contents of the repository are visible in the file browser in JupyterLab, or run the **ls** command in the terminal to verify that the repository shows as a directory.

4.3.2. Uploading an existing notebook file to JupyterLab from a Git repository by using the CLI

You can use the command line interface to clone a Git repository into your workspace to continue your work or integrate files from an external project.

Prerequisites

- You have a launched and running workbench based on a JupyterLab image.

Procedure

1. Copy the HTTPS URL for the Git repository.
   - In GitHub, click **Code → HTTPS** and then click the **Copy URL to clipboard** icon.
   - In GitLab, click **Code** and then click the **Copy URL** icon under **Clone with HTTPS**.
2. In JupyterLab, click **File → New → Terminal** to open a terminal window.
3. Enter the **git clone** command:

   ```
   git clone <git-clone-URL>
   ```

   Replace **git-clone-URL>** with the HTTPS URL, for example:

   ```
   [1234567890@jupyter-nb-jdoe ~]$ git clone https://github.com/example/myrepo.git
   Cloning into myrepo...
   remote: Enumerating objects: 11, done.
   remote: Counting objects: 100% (11/11), done.
   remote: Compressing objects: 100% (10/10), done.
   remote: Total 2821 (delta 1), reused 5 (delta 1), pack-reused 2810
   Receiving objects: 100% (2821/2821), 39.17 MiB | 23.89 MiB/s, done.
   Resolving deltas: 100% (1416/1416), done.
   ```

Verification

- Check that the contents of the repository are visible in the file browser in JupyterLab, or run the **ls** command in the terminal to verify that the repository shows as a directory.

4.3.3. Updating your project with changes from a remote Git repository

You can pull changes made by other users into your data science project from a remote Git repository.

Prerequisites

- You have a launched and running workbench based on a JupyterLab image.
- You have credentials for logging in to Jupyter.

- You have configured the remote Git repository.
- You have permissions to pull files from the remote Git repository to your local repository.
- You have imported the Git repository into JupyterLab, and the contents of the repository are visible in the file browser in JupyterLab.

Procedure

1. In the JupyterLab interface, click the **Git** button ().
2. Click the **Pull latest changes** button ().

Verification

- You can view the changes pulled from the remote repository on the **History** tab in the Git pane.

4.3.4. Pushing project changes to a Git repository

To build and deploy your application in a production environment, upload your work to a remote Git repository.

Prerequisites

- You have opened a Jupyter notebook in the JupyterLab interface.
- You have added the relevant Git repository to your workbench.
- You have permission to push changes to the relevant Git repository.
- You have installed the Git version control extension.

Procedure

1. Click **File → Save All** to save any unsaved changes.
2. Click the Git icon () to open the Git pane in the JupyterLab interface.
3. Confirm that your changed files appear under **Changed**.
   If your changed files appear under **Untracked**, click **Git → Simple Staging** to enable a simplified Git process.
4. Commit your changes.
   a. Ensure that all files under **Changed** have a blue checkmark beside them.
   b. In the **Summary** field, enter a brief description of the changes you made.
   c. Click **Commit**.
5. Click **Git → Push to Remote** to push your changes to the remote repository.
6. When prompted, enter your Git credentials and click **OK**.

Verification

- Your most recently pushed changes are visible in the remote Git repository.

4.4. MANAGING PYTHON PACKAGES

In JupyterLab, you can view the Python packages that are installed on your workbench image and install additional packages.

4.4.1. Viewing Python packages installed on your workbench

You can check which Python packages are installed on your workbench and which version of the package you have by running the **pip** tool in a notebook cell.

Prerequisites

- Log in to JupyterLab and open a Jupyter notebook.

Procedure

1. Enter the following in a new cell in your Jupyter notebook:

```
!pip list
```

2. Run the cell.

Verification

- The output shows an alphabetical list of all installed Python packages and their versions. For example, if you use the **pip list** command immediately after creating a workbench that uses the **Minimal** image, the first packages shown are similar to the following:

```
Package                       Version
----------------------------- ----------
aiohttp                        3.7.3
alembic                         1.5.2
appdirs                        1.4.4
argo-workflows                    3.6.1
argon2-cffi                   20.1.0
async-generator                    1.10
async-timeout                  3.0.1
attrdict                     2.0.1
attrs                       20.3.0
backcall                       0.2.0
```

Additional resources

- Installing Python packages on your workbench

4.4.2. Installing Python packages on your workbench

You can install Python packages that are not part of the default workbench by adding the package and the version to a **requirements.txt** file and then running the **pip install** command in a notebook cell.

NOTE

Although you can install packages directly, it is recommended that you use a **requirements.txt** file so that the packages stated in the file can be easily re-used across different workbenches.

Prerequisites

- Log in to JupyterLab and open a Jupyter notebook.

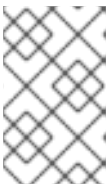
Procedure

1. Create a new text file using one of the following methods:
   - Click **+** to open a new launcher and then click **Text file**.
   - Click **File → New → Text File**.
2. Rename the text file to **requirements.txt**.
   a. Right-click the name of the file and then click **Rename Text**. The **Rename File** dialog opens.
   b. Enter **requirements.txt** in the **New Name** field and then click **Rename**.
3. Add the packages to install to the **requirements.txt** file.

```
altair
```

   You can specify the exact version to install by using the **==** (equal to) operator, for example:

```
altair==4.1.0
```



   NOTE

   Red Hat recommends specifying exact package versions to enhance the stability of your workbench over time. New package versions can introduce undesirable or unexpected changes in your environment's behavior.

   To install multiple packages at the same time, place each package on a separate line.
4. Install the packages in **requirements.txt** to your server by using a notebook cell.
   a. Create a new notebook cell and enter the following command:

```
!pip install -r requirements.txt
```

   b. Run the cell by pressing Shift and Enter.

IMPORTANT

The **pip install** command installs the package on your workbench. However, you must run the **import** statement in a code cell to use the package in your code.

```
import altair
```

Verification

- Confirm that the packages in the **requirements.txt** file appear in the list of packages installed on the workbench. See Viewing Python packages installed on your workbench for details.

4.5. UPDATING WORKBENCH SETTINGS BY RESTARTING YOUR WORKBENCH

You can update the settings on your workbench by stopping and relaunching the workbench. For example, if your server runs out of memory, you can restart the server to make the container size larger.

Prerequisites

- A running workbench.
- Log in to JupyterLab.

Procedure

1. Click **File → Hub Control Panel**.
   The **Workbench control panel** opens.
2. Click the **Stop workbench** button.
   The **Stop server** dialog opens.
3. Click **Stop server** to confirm your decision.
   The **Start a basic workbench** page opens.
4. Update the relevant workbench settings and click **Start workbench**.

Verification

- The workbench starts and contains your updated settings.